agent_loop_demo

Converted from Jupyter Notebook

AI Agent Loop Demo

This notebook demonstrates the core mechanism behind an AI agent: a model that can decide to call a tool, receive the result, and continue reasoning toward a goal — all in a loop.

This notebook contains starter code. Please fill in details as needed. ---

Setup - Gemini API key

How to get the key (takes about two minutes):

• Go to [aistudio.google.com](https://aistudio.google.com/prompts/new_chat)

• Sign in with your Google account

• Click "Get API key" in the left sidebar

• Click "Create API key" — choose "Create API key in new project" if prompted

Copy the key that appears (it starts with AIza...)

> The free tier allows up to 10 requests per minute and 250 requests per day, which is enough to work through this notebook.

If you see occasional slow-downs when you run several questions back to back — that is normal, and the code handles it automatically.

References

• [Google agent loop](https://github.com/xiaosuhu/midas-ai-in-research/blob/v1.0-dev/docs/notebooks/agent_loop_demo.ipynb$0)

• https://ai.google.dev/gemini-api/docs/thinking#signatures

• https://ai.google.dev/gemini-api/docs/text-generation$0

• [Developer guide](https://github.com/xiaosuhu/midas-ai-in-research/blob/v1.0-dev/docs/notebooks/agent_loop_demo.ipynb$0)

Code cell 3

```
import os

# NEVER share API keys directly. Save them in the environment.
try:
    from google.colab import userdata
    os.environ['GOOGLE_API_KEY'] = userdata.get('GOOGLE_API_KEY')
    print('API key loaded from Colab Secrets.')

# Fall back to manual entry if Secrets are not set up.
# REMOVE API keys before pushing code to Github. This fallback helps.
except Exception:
    import getpass
    key = getpass.getpass('Paste your Gemini API key and press Enter: ')
    os.environ['GOOGLE_API_KEY'] = key
    print('API key set.')
```

Code cell 4

```
# Quiet ensures your UI is not full of logs.
!pip install google-generativeai --quiet
print("Done.")
```

Code cell 5

```
# @title User input needed.

# TODO: Fill in documents in the format mentioned in the first example.
DOCUMENTS = {
    "sample_protocol": (
        "Students must fill in documents in this section."
        "Documents can be related to any topic - sports, manga, food, music etc."
        "Have a minimum of 4 sentences with each document."
        "Duplicate this for a minimum of 4 documents to test your agent."
    ),
    "fill_in1":(),
    "fill_in2":(),
    "fill_in3":(),
    "fill_in4":()
}
```

Code cell 6

```
# @title Define a tool
def search_documents(query: str) -> str:
    """Search the document collection for paragraphs relevant to the query."""
    query_lower = query.lower()
    results = []

    for doc_name, content in DOCUMENTS.items():
        words = set(query_lower.split())
        match_count = sum(1 for word in words if word in content.lower())
        if match_count > 0:
            results.append((match_count, doc_name, content))

    results.sort(reverse=True)

    if not results:
        return "No relevant documents found for that query."

    # Return the top two matches
    output = []
    for _, name, content in results[:2]:
        output.append(f"[Source: {name}]\n{content}")
    return "\n\n".join(output)


# Sanity check for regex match.
print(search_recipes("minimum of 4"))
```

Code cell 7

```
# @title Ask gemini to search for something
import google.generativeai as genai
import time

genai.configure(api_key=os.environ["GOOGLE_API_KEY"])

model = genai.GenerativeModel(
    # This will break.
    # TODO: Find the right model to call in this cell from developer docs.
    model_name="claude",
    tools=[search_recipes],
    system_instruction=(
        '''
        You are a chef training at a Michelin star restaurant.
        When you need specific information about temperatures,
        ingredients to use, cook time etc. use the search_documents tool.
        Only answer based on what you find in the documents.
        If the documents do not contain the answer, say so clearly.
        '''
    )
)

print("Model ready:", model.model_name)
```

Code cell 8

```
import time
from google import genai
from google.genai import types

client = genai.Client()  # reads GEMINI_API_KEY / GOOGLE_API_KEY from env

search_tool = types.Tool(function_declarations=[
    types.FunctionDeclaration(
        name="search_documents",
```

```
        description="Search the documents to find most the relevant paragraph.",
        parameters={
            "type": "object",
            "properties": {"query": {"type": "string"}},
            "required": ["query"],
        },
    )
])

config = types.GenerateContentConfig(
    tools=[search_tool],
    automatic_function_calling=types.AutomaticFunctionCallingConfig(disable=True),
)


def send_with_retry(contents, max_retries=3):
    for attempt in range(max_retries):
        try:
            return client.models.generate_content(
                model=model.model_name,
                contents=contents, config=config
            )
        except Exception as e:
            if "429" in str(e) and attempt < max_retries - 1:
                wait = 20 * (attempt + 1)
                print(f"  Rate limit hit, waiting {wait}s...")
                time.sleep(wait)
            else:
                raise


def run_agent(question: str, max_steps: int = 8) -> str:
    print(f"Question: {question}\n" + "-" * 50)
    contents = [types.Content(role="user", parts=[types.Part(text=question)])]

    for step in range(1, max_steps + 1):
        response = send_with_retry(contents)
        model_content = response.candidates[0].content

        # Append the model's turn UNMODIFIED, so thought signatures are preserved
        contents.append(model_content)

        calls = [p.function_call for p in model_content.parts if p.function_call]
        if not calls:
            print(f"\nFinal answer:\n{response.text}")
            return response.text

        # Run every requested tool call, reply with all results in ONE user turn
        result_parts = []
        for call in calls:
            args = dict(call.args)
            print(f"Step {step}: model calls {call.name}({args})")
            result = search_recipes(**args)
            print(f"  Result preview: {str(result)[:100]}...")
            result_parts.append(
                types.Part.from_function_response(
                    name=call.name, response={"result": result}
                )
            )
        contents.append(types.Content(role="user", parts=result_parts))

    return "(Stopped: hit max_steps)"


_ = run_agent("What topics are students supposed to share documents on?")
```

Code cell 9

```
# @title Ask follow up questions

# TODO: Enter questions based on documents for the agent to answer.
questions = [
    "Add a question here.",
    # This question is not answered by the database.
    "How many ghosts eat carrots everyday?",
    # This question is part of the general knowledge of the model.
    "What is the most popular food item in Canada?"
]

for i, q in enumerate(questions):
    if i > 0:
        time.sleep(15)  # stay within the free-tier rate limit.
```

```
    print("\n" + "=" * 60)
    run_agent(q)
```

Code cell 10

```
# This cell will check if you have added atleast 2 more questions.
assert(len(questions) >= 4)
# This will check if you added a question.
assert("Add a question here." not in questions)
```